

32. 実測勾配推定ノード

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/grade_node.py
種別	Python
区分	runtime node
役割	distance と altitude/GPS から grade を推定し /vehicle/grade へ publish する。
起動文脈	measure/live の監視・記録用の grade source。

このファイルを読む理由

distance と altitude/GPS から grade を推定し /vehicle/grade へ publish する。

- 現在の solar mpc_node は route profile 勾配を主に使い、/vehicle/grade を直接は使わない。
- ただし logger や補助用途では重要。

呼び出し関係

どこから呼ばれるか

- launch/solar_measurement.launch.py
- mpc_solarcar/live_launch.py

次に読むと理解が進むファイル

- 特記事項なし。

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from sensor_msgs.msg import NavSatFix

class GradeNode(Node):
    def __init__(self):
        super().__init__('grade_node')
        self.declare_parameter('min_speed_kmh', 5.0)
        self.declare_parameter('altitude_alpha', 0.2)
        self.declare_parameter('min_delta_s_km', 0.01)
        self.declare_parameter('gps_topic', '/sim/gps')
        self.declare_parameter('altitude_topic', '/vehicle/altitude_m')

        self.min_speed_kmh = float(self.get_parameter('min_speed_kmh').value)
        self.alpha = float(self.get_parameter('altitude_alpha').value)
        self.min_delta_s_km = float(self.get_parameter('min_delta_s_km').value)
        self.gps_topic = self.get_parameter('gps_topic').value
        self.altitude_topic = self.get_parameter('altitude_topic').value

        self.altitude_m = math.nan
        self.altitude_ema = math.nan
        self.s_km = math.nan
        self.speed_kmh = math.nan
```

主要構造の読み取り

主要クラスは GradeNode。主要関数は main。ROS パラメータ宣言は 5 件。ROS I/O は publisher 1 件、subscription 4 件。

主要クラス・関数

行	種別	名前	補足
10	class	GradeNode	bases: Node
11	function	__init__	args: self
43	function	_on_s_km	args: self, msg
46	function	_on_speed	args: self, msg
49	function	_on_altitude	args: self, msg
52	function	_on_gps	args: self, msg
56	function	_update_altitude	args: self, altitude_m
63	function	_step	args: self
89	function	main	

ファイルを上から読んだときの定義順

行	内容
10	クラス GradeNode を定義する。
89	関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import math</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L25, L26, L27, L28, L31, L53, L58, L64, ...。
2	<code>import time</code>	<code>wall/monotonic time</code> に基づく周期制御や <code>freshness</code> 判定を行うため。このファイル内での主な使用位置は L32。
4	<code>import rclpy</code>	ROS 2 Python ノードとして起動・ <code>spin</code> するため。このファイル内での主な使用位置は L90, L92, L94。
5	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L10。
6	<code>from std_msgs. msg import Float32</code>	<code>Float32</code> や <code>String</code> など軽量メッセージで <code>planner / vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L34, L35, L36, L39, L43, L46, L49, L65, ...。
7	<code>from sensor_msgs. msg import NavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L37, L52。

関数・クラスを上から順に解説

L10 クラス `GradeNode`

- 定義: `GradeNode(bases=Node)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `__init__`, `_update_altitude`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `get_logger`, `get_parameter`, `info`, `isfinite`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. 関数 `__init__` を定義する。
2. 関数 `_on_s_km` を定義する。
3. 関数 `_on_speed` を定義する。
4. 関数 `_on_altitude` を定義する。
5. 関数 `_on_gps` を定義する。
6. 関数 `_update_altitude` を定義する。
7. 関数 `_step` を定義する。

L89 関数 `main`

- 定義: `main()`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `GradeNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `rcipy.init(...)` を実行する。
2. `node` に `GradeNode()` の結果を代入する。
3. `rcipy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rcipy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
13	<code>min_speed_kmh</code>	5.0
14	<code>altitude_alpha</code>	0.2
15	<code>min_delta_s_km</code>	0.01
16	<code>gps_topic</code>	/sim/gps
17	<code>altitude_topic</code>	/vehicle/altitude_m

ROS topic I/O

行	種別	topic	callback / 補足
39	publisher	/vehicle/grade	publisher
34	subscription	/vehicle/s_km	<code>self._on_s_km</code>
35	subscription	/vehicle/speed_kmh	<code>self._on_speed</code>
36	subscription	<code>self.altitude_topic</code>	<code>self._on_altitude</code>
37	subscription	<code>self.gps_topic</code>	<code>self._on_gps</code>

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/grade_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。